

1. **Single-page apps and URLs:** Multi-page sites and single-page apps · The History API: changing the URL without a request · Refresh and deep links: every path must return index.html · Hash URLs and path URLs
2. **Routes, links and matching:** Setting up a router (React Router) · Routes: mapping a URL to a component · `<Link>` instead of `<a href>` · How the router picks a route: ranking, not order · URL parameters: `useParams` (always strings) · Query strings: `useSearchParams`
3. **Nested routes and layouts:** Nested routes and layouts: `<Outlet />` · Index routes: the default child · The catch-all route: a 404 page
4. **Navigating from code:** Navigating from code: `useNavigate` · Push or replace: what the Back button will do
5. **Protected routes and data:** Protected routes: a guard component · Back to where you were after log-in · Loading data for a route

Single-page apps and URLs

Multi-page sites and single-page apps

In a **multi-page site** every link loads a new HTML page from the server; in a **single-page app (SPA)** the server sends one HTML page once, and the app then swaps screens and changes the URL itself, asking the server only for data.

What happens when the user clicks "Task 7"

	Multi-page site	Single-page app
Request	GET /tasks/7: a whole HTML page	None for the page; GET /api/tasks/7: JSON (see Fetching data)
Who builds the screen	The server	The app, in the browser
JavaScript state (variables, forms)	Lost	Kept
The URL changes because...	The browser loaded a new document	The router called <code>history.pushState</code>
Back button	The browser loads the previous page	The router draws the previous screen
First visit	Fast: one ready-made page	Slower: the bundle must download and run first

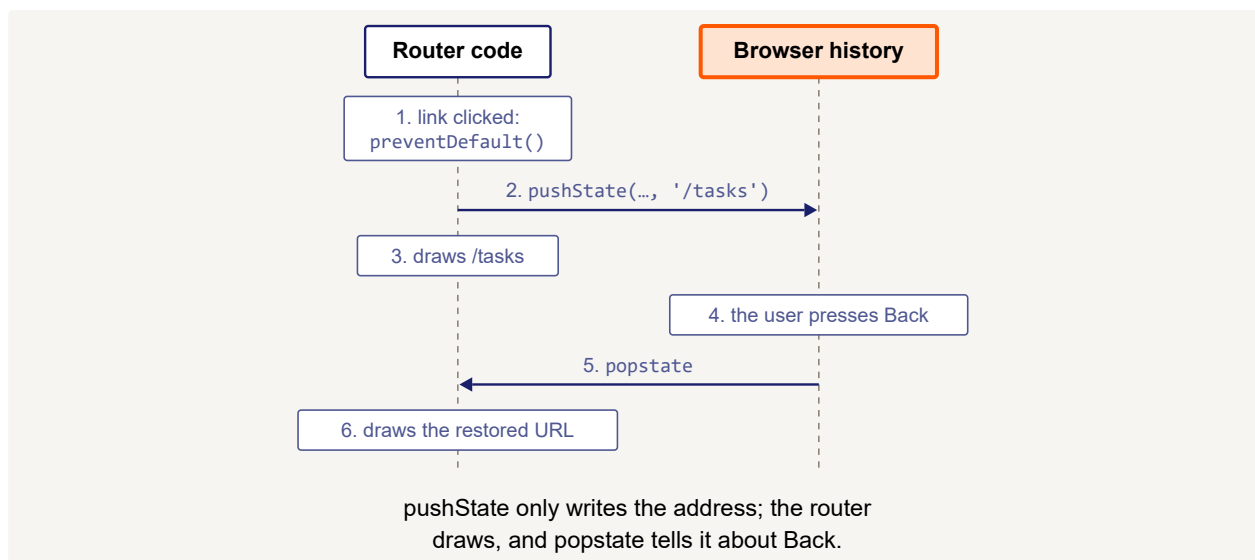
Example. Open the Network tab (see [Watching the conversation: the Network tab](#)) on a React app and click around: the "Doc" filter shows one request, the first; "Fetch/XHR" shows the JSON calls each screen makes. On a classic multi-page site every click adds a new "Doc" request.

Common mistake. Thinking a single-page app has a single URL. A good SPA has as many URLs as screens (`/tasks`, `/tasks/7`, `/tasks/7/edit`); "single page" means the one HTML document the server sends, not the address bar.

The History API: changing the URL without a request

The browser's **History API** lets JavaScript change the URL without loading a page: `pushState` adds a history entry, `replaceState` overwrites the current one, and the `popstate` event fires on Back and Forward. Every client-

side router is built on them.



history.pushState(state, '', '/tasks') Adds an entry and moves the pointer to it; the address bar shows /tasks. It does **not** fire popstate, so the router renders right after calling it.

history.replaceState(...) Overwrites the current entry, for redirects (see [Push or replace](#)).

popstate Fires on window when Back or Forward moves the pointer; the router reads location.pathname and draws that screen.

history.back(), history.go(-2) Move the pointer from code.

```

// A hand-made router in plain JavaScript: what a library like React Router does underneath
const routes = {
  '/': () => '<h1>Home</h1>',
  '/tasks': () => '<h1>All tasks</h1>',
};

function render() {
  const page = routes[location.pathname] || (() => '<h1>Not found</h1>');
  document.querySelector('#app').innerHTML = page();
}

// 1. Links: change the URL without a request, then draw
document.addEventListener('click', (e) => {
  const a = e.target.closest('a[data-link]');
  if (!a) return;
  // stop the browser loading a page
  e.preventDefault();
  history.pushState(null, '', a.getAttribute('href'));
  // pushState does not fire popstate
  render();
});

// 2. Back and Forward: the browser fires popstate, we draw the URL it restored
window.addEventListener('popstate', render);

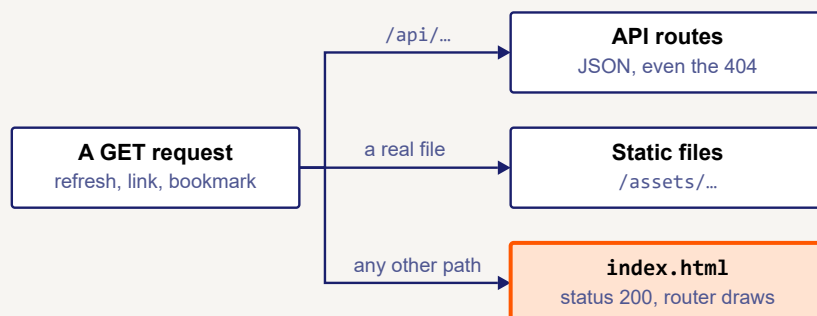
render();
// → 3. the first load
  
```

Example. With this hand-made router, on / the user clicks : the listener cancels the navigation, pushState makes the history [/, /tasks], and render() draws "All tasks". Back moves the pointer to /, fires popstate, and render() draws "Home". No request reached the server.

Common mistake. Expecting `pushState` to show the new page, or to fire `popstate`. It only changes the address bar and the history; without a render afterwards, the URL says `/tasks` while the screen still shows the home page.

Refresh and deep links: every path must return `index.html`

A client-side route such as `/tasks/7` exists only inside the app: on a **refresh** or a shared link the browser asks the **server** for `/tasks/7`, so the server must answer every app path with `index.html` (the **SPA fallback**), or the user gets a 404.



The API answers JSON, files are files, and every other path gets the app.

Where the fallback is set up

Vite dev server, vite preview Built in, which is why the problem only appears after deploying.

Netlify, Vercel, Nginx A rewrite rule: `/* /index.html 200` in Netlify's `_redirects`, a Vercel rewrite, `try_files $uri /index.html` in Nginx.

GitHub Pages Cannot rewrite: copy `index.html` to `404.html` (the app works, but deep links answer with status 404), or use hash URLs.

Express One last route after the API and the static files, as in the code.

```

// Express serving the built React app (the dist/ folder from npm run build)
const path = require('path');
const dist = path.join(__dirname, '..', 'client', 'dist');

app.use('/api', apiRouter);
// → 1. the API first
app.use(express.static(dist));
// → 2. real files: /assets/index-3f2a.js ...

// 3. the SPA fallback: any other GET that is not /api/... gets index.html
app.get(/^\/(?!api\/).*/, (req, res) => {
  res.sendFile(path.join(dist, 'index.html'));
});

app.use((req, res) => res.status(404).json({ error: 'Not found' }));
// → 4. unknown /api/... routes
  
```

Example. In the router simulator, switch off "The server answers every path with `index.html`", click a link to `/tasks` (it works: no request is made), then press **Refresh**: the page becomes the server's 404 Not Found · Cannot GET `/tasks`. Switch the fallback on and refresh again: the app starts and draws `/tasks`.

Common mistake. Making the fallback catch everything, API included: GET `/api/tsks` (a typo) then answers 200 with an HTML page, and the front end fails with "Unexpected token '<' in JSON". Exclude `/api/` from the fallback, so unknown API paths keep their JSON 404 (see The 404 catch-all).

Hash URLs and path URLs

A router can keep the screen in the **path** (`/tasks/7`, which needs the server fallback) or in the **hash** (`/#/tasks/7`); the hash is never sent to the server, so hash URLs work on any static host, at the price of uglier URLs.

The same screen, two kinds of URL

	Path: /tasks/7	Hash: /#/tasks/7
What the server receives	GET /tasks/7	GET / (the hash is not sent)
Server configuration	SPA fallback needed	None
Browser API underneath	pushState + popstate	location.hash + hashchange
React Router	BrowserRouter / createBrowserRouter	HashRouter / createHashRouter
In-page anchors (#section)	Work as usual	Clash with the route

```
// A hash router in plain JavaScript: no server configuration needed
function render() {
  const path = location.hash.slice(1) || '/';
  // → '#/tasks/7' → '/tasks/7'
  document.querySelector('#app').textContent = 'Screen for ' + path;
}
// links: <a href="#/tasks/7">
window.addEventListener('hashchange', render);
render();
```

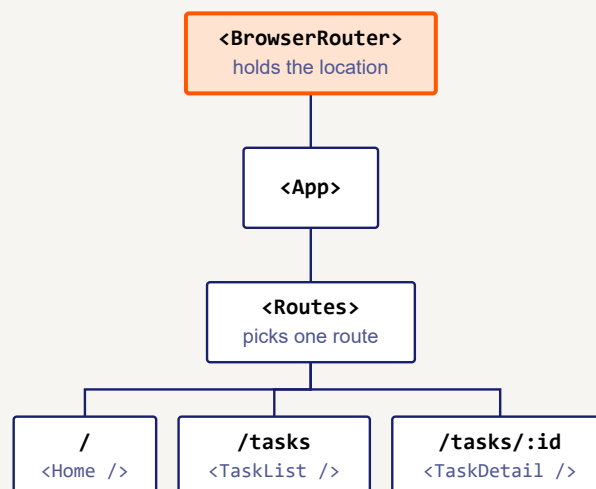
Example. Deploy the same app twice to a static host without rewrites. With `BrowserRouter`, `https://me.github.io/app/tasks/7` gives a 404 on refresh. With `HashRouter`, `https://me.github.io/app/#/tasks/7` always works: the server only ever sees `/app/`.

Common mistake. Switching to `HashRouter` while keeping `href="/tasks"` in plain `<a>` tags. Those still point at a path, so they load a page the host does not have. Use the router's `<Link to="/tasks">`, which writes the right URL for the router you chose (see [Link instead of a href](#)).

Routes, links and matching

Setting up a router (React Router)

A router is a component at the top of the app that listens to the URL and lets every component below it read the location; with **React Router**: `npm install react-router`, wrap the app in `<BrowserRouter>`, and list the screens with `<Routes>` and `<Route>`.



One router at the top; every component below it can read the URL.

`useLocation()`

The current location: pathname (`/tasks/7`), search (`?page=2`), hash, and state, data attached to this history entry by the navigation that created it.

`react-router` OR `react-router-dom`

Version 7 is installed as `react-router`; version 6, in many tutorials and existing projects, was `react-router-dom`, with the same component and hook names. Import from whichever package .json lists.

JSX routes or route objects

`<BrowserRouter>` + `<Routes>`, used on these cards; or `createBrowserRouter([...])` + `<RouterProvider>`, the "data" mode,

which adds loader functions that fetch before a route renders. Matching, nesting, params and links work the same.

```
// main.jsx: one router, at the top
import { createRoot } from 'react-dom/client';
// version 6: 'react-router-dom'
import { BrowserRouter } from 'react-router';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);

// App.jsx: which component renders for which URL
import { Routes, Route } from 'react-router';
import Home from './pages/Home.jsx';
import TaskList from './pages/TaskList.jsx';
import TaskDetail from './pages/TaskDetail.jsx';

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/tasks" element={<TaskList />} />
      <Route path="/tasks/:id" element={<TaskDetail />} />
    </Routes>
  );
}

// The same routes as objects (data mode)
// const router = createBrowserRouter([
//   { path: '/', element: <Home /> },
//   { path: '/tasks', element: <TaskList /> },
//   { path: '/tasks/:id', element: <TaskDetail /> },
// ]);
// createRoot(root).render(<RouterProvider router={router} />);
```

Example. A typical layout of files: `src/pages/` holds one component per screen (`TaskList.jsx`, `TaskDetail.jsx`, `Login.jsx`), `src/components/` the pieces they share (`TaskItem`, `NavBar`), and `App.jsx` the route table that connects URLs to pages. Reading `App.jsx` alone tells you every screen the app has.

Common mistake. Calling `useNavigate()` in a component rendered **outside** the router, such as a provider that keeps the logged-in user (see [An authentication context](#)) placed above `<BrowserRouter>` in `main.jsx`. React Router throws "useNavigate() may be used only in the context of a `<Router>` component". Put the router at the very top, and any provider that navigates inside it.

Routes: mapping a URL to a component

A route pairs a URL pattern (`path="/tasks/:id"`) with what to render (`element={<TaskDetail />}`); `<Routes>` picks the best match for the current URL, renders its element, and chooses again whenever the URL changes.

tasks A literal segment: must be equal.
:id Any one non-empty segment (see [URL parameters](#)).
*** at the end** Whatever is left of the URL.

- **Ignored when matching:** letter case, a trailing slash, the query string and the hash:
`/Tasks/7/?tab=notes` matches `/tasks/:id`.
- **No match:** React Router renders nothing and only logs a warning in the console.
- **element takes an element**, `<TaskList />`, not the function `TaskList`, so it can carry props or a wrapper. It renders only while its route matches; navigating away unmounts it and its state is lost (see [Nested routes and layouts](#)).

```
import { useState } from 'react';

// A hand-made router, to see the idea: the URL lives in state,
// and a table of routes says what to render for it.
// (React Router reads the real URL; these boxes cannot change it.)
const routes = [
  { path: '/', element: <h2>Home</h2> },
  { path: '/tasks', element: <h2>All tasks</h2> },
  { path: '/about', element: <h2>About this app</h2> },
];

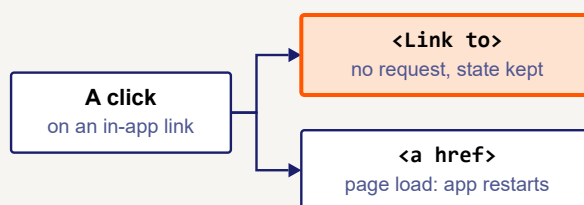
function App() {
  const [path, setPath] = useState('/');
  const route = routes.find((r) => r.path === path.toLowerCase().replace(/(.)\$/, '$1'));
  return (
    <>
      <nav style={{ display: 'flex', flexWrap: 'wrap', gap: 6 }}>
        {[ '/', '/tasks', '/about', '/Tasks/', '/nope' ].map((p) => (
          <button key={p} onClick={() => setPath(p)} aria-pressed={p === path}>{p}</button>
        ))}
      </nav>
      <p>URL: <code>{path}</code></p>
      {route ? route.element : <p>(nothing: no route matches)</p>}
    </>
  );
}
```

Example. In the Try it box, click /Tasks/: it still shows "All tasks", because the toy matcher, like React Router, ignores case and a trailing slash. Click /nope: no route matches and nothing renders, which is what React Router does too (plus a console warning No routes matched location "/nope").

Common mistake. Writing routes in the React Router 5 style of old tutorials: `<Route exact path="/" component={Home} />` inside a `<Switch>`. In current versions `Switch` does not exist (it is `Routes`), `exact` is gone (every path is exact unless it ends in `/*`) and `component` is ignored, so the route renders nothing. Write `<Route path="/" element={Home} />`.

<Link> instead of <a href>

Inside the app, navigate with `<Link to="/tasks">`: it renders a normal `<a href>` (accessible, can open in a new tab), but the router handles the click, so there is no page load and the app keeps its state; a plain `<a href="/tasks">` reloads the whole app.



Same click, two outcomes: the router keeps the app; a plain link restarts it.

- **Still a real link:** it renders an `<a href>`, so Ctrl/Cmd-click opens a new tab, the URL shows on hover, screen readers announce a link and Tab reaches it.
- **Buttons for actions, links for places:** save, delete and open a dialog are buttons; going somewhere is a link.
- `<NavLink>` knows whether it points at the current page: it adds the class `active` and `aria-current="page"`. It is active for its URL **and everything below** (`/tasks` at `/tasks/7`); end makes it exact, which the link to `/` needs.
- **Other sites** stay plain `<a href="https://...">`.

```
import { Link, NavLink } from 'react-router';

function NavBar() {
```

```

return (
  <nav aria-label="Main">
    <NavLink to="/" end>Home</NavLink>
    <NavLink to="/tasks">Tasks</NavLink>
    <NavLink to="/tasks/new" className={({ isActive }) => (isActive ? 'tab current' : 'tab')}>
      New task
    </NavLink>
  </nav>
);
}

// In a list: a link per item, built from the data
<li key={task.id}><Link to={"/tasks/" + task.id}>{task.title}</Link></li>

/* CSS: highlight the current page */
/* nav a.active, nav a[aria-current="page"] { font-weight: 700; border-bottom: 3px solid; } */

```

```

import { useState } from 'react';

// Simulated: a full page load is a new start of the app,
// imitated here by giving the app a new key (React remounts it from zero).
function TheApp({ path, go, fullLoad }) {
  // state kept in memory
  const [clicks, setClicks] = useState(0);
  return (
    <>
      <p>State in memory: <strong>{clicks}</strong>{' '}</p>
      <button onClick={() => setClicks(clicks + 1)}>+1</button></p>
      <p>
        <button onClick={() => go('/tasks')}>{'<Link to="/tasks">'}</button>{' '}</p>
        <button onClick={() => fullLoad('/tasks')}>{'<a href="/tasks">'}</button>{' '}</p>
        <button onClick={() => go('/')}>{'<Link to="/">'}</button>
      </p>
      <p>Page: <code>{path}</code></p>
    </>
  );
}

function App() {
  const [path, setPath] = useState('/');
  const [loads, setLoads] = useState(1);
  function fullLoad(to) {
    setPath(to);
    // the browser starts the app again
    setLoads(loads + 1);
  }
  return (
    <>
      <p>Page loads (requests for index.html): {loads}</p>
      <TheApp key={loads} path={path} go={setPath} fullLoad={fullLoad} />
    </>
  );
}

```

Example. In the Try it box, press +1 a few times, then use the two "Link" buttons: the page changes and the counter stays. Press the <a href> button: "Page loads" goes up and the counter is back to 0, because the whole app started again, which is what a real <a href> inside an SPA does to all its state.

Common mistake. Using <button onClick={() => navigate('/tasks/' + id)}> for plain navigation in a list. It works with a mouse, but the user cannot open the task in a new tab or see where it leads, and assistive technology announces a button, not a link. If clicking it goes somewhere, it is a <Link>.

How the router picks a route: ranking, not order

When several routes match a URL, React Router does not take the first one written: it **rank**s them, and the most specific path wins (a literal segment beats a :param, which beats *), so /tasks/new and /tasks/:id can be written

in any order.

Scores for some paths (React Router 6 and 7)

Path	How the score is made	Score
/tasks/:id/edit	4 segments + 1 + 10 + 3 + 10	28
/tasks/new	3 segments + 1 + 10 + 10	24
/tasks/:id	3 segments + 1 + 10 + 3	17
/tasks (index child)	3 segments + 2 (index) + 1 + 10 + 1	17
/tasks	2 segments + 1 + 10	13
/	2 segments + 1 + 1	4
*	2 segments – 2 (splat) + 1	1

Example. In the router simulator (example "A task app"), type `/tasks/new` and click the Link: the ranking table at the bottom shows `/tasks/:id/edit` (28) tried and rejected, then `/tasks/new` (24) matching; `/tasks/:id` (17) is never tried. Challenge 2 asks you to predict this, and challenge 6 shows the one case where order decides.

Common mistake. Carrying the Express habit over and worrying that `:id` "swallows" `new`, then renaming the route to `/new-task` or adding `if (id === 'new')` checks in `TaskDetail`. React Router already picks `/tasks/new`; the workaround only adds confusion.

URL parameters: useParams (always strings)

A `:name` segment in a route path captures that part of the URL; inside the route's components, `useParams()` returns the captured values as an object of strings: `{ id: '7' }` for `/tasks/7`.

- **Always strings**, already decoded (`%20` becomes a space): `id === 7` is false when `id` is `'7'`. Convert with `Number(id)` and check the result: `/tasks/abc` also matches, giving `NaN`.
- **One non-empty segment**: `/tasks/:id` matches neither `/tasks/` nor `/tasks/7/edit`.
- **Several params** are fine (`/users/:userId/tasks/:taskId`); a layout route also sees the params of the child that matched below it, and a splat puts the rest of the URL in `params['*']`.
- **Name them** after what they hold (`:taskId`, not `:x`). The server reads the same parts with `req.params` (see [Route parameters](#)).

```
// App.jsx
<Route path="/tasks/:id" element={<TaskDetail />} />

// TaskDetail.jsx
import { useParams } from 'react-router';

export default function TaskDetail() {
  const { id } = useParams();
  // → '7' for /tasks/7: a string
  const taskId = Number(id);
  if (!Number.isInteger(taskId) || taskId < 1) return <p>There is no task "{id}".</p>;
  return <h1>Task {taskId}</h1>;
}
```

```
import { useState } from 'react';

// A hand-made matcher for one pattern, to see what useParams() returns.
function matchPath(pattern, path) {
  const want = pattern.split('/');
  const got = path.split('/');
  if (want.length !== got.length) return null;
  const params = {};
  for (let i = 0; i < want.length; i++) {
    if (want[i].startsWith(':')) {
      // a param needs a non-empty segment
      if (!got[i]) return null;
      try { params[want[i].slice(1)] = decodeURIComponent(got[i]); } catch { return null; }
    }
  }
}
```

```

    } else if (want[i] !== got[i]) return null;
  }
  return params;
}

function App() {
  const [path, setPath] = useState('/tasks/7');
  const params = matchPath('/tasks/:id', path);
  return (
    <>
      <label>URL <input value={path} onChange={(e) => setPath(e.target.value)} /></label>
      <p>Pattern: <code>/tasks/:id</code></p>
      {params ? (
        <ul>
          <li>params: <code>{JSON.stringify(params)}</code></li>
          <li>typeof params.id: <code>{typeof params.id}</code></li>
          <li>params.id === 7: <code>{String(params.id === 7)}</code></li>
          <li>Number(params.id) === 7: <code>{String(Number(params.id) === 7)}</code></li>
        </ul>
      ) : <p>No match: the URL has a different shape.</p>}
    </>
  );
}

```

Example. In the Try it box, the URL `/tasks/7` gives `{ "id": "7" }`: `typeof` is string, `params.id === 7` is false and `Number(params.id) === 7` is true. Type `/tasks/buy%20milk`: the param is decoded to buy milk. Type `/tasks/7/edit` or `/tasks/`: no match, the shape is different.

Common mistake. Comparing the param with a number from the data: `tasks.find((t) => t.id === id)` finds nothing, because `t.id` is the number 7 and `id` is the string '7', so the page says "not found" for a task that exists. Convert once, `const taskId = Number(id)`, and compare numbers.

Query strings: useSearchParams

The query string (`?done=false&page=2`) holds the optional settings of a screen (filters, search text, sort, page); React Router does not use it to choose a route, and the page reads and changes it with `useSearchParams()`.

- **Reading:** `const [searchParams, setSearchParams] = useSearchParams()`. `searchParams` is a standard `URLSearchParams`; `get('page')` returns a **string**, or null when absent, so default and convert: `Number(searchParams.get('page') ?? 1)`.
- **Writing:** `setSearchParams({ done: 'false', page: '2' })` replaces the whole query and navigates (a new history entry). For keystrokes pass `{ replace: true }` as a second argument, so Back is not one entry per letter.
- **Changing one key:** start from a copy, `setSearchParams((prev) => { const next = new URLSearchParams(prev); next.set('page', '2'); return next; })`.
- **A new filter resets the page to 1** (page 3 of the old results may not exist), and the query values go in the dependencies of the effect that fetches (see [Pagination controls](#)).

```

import { useState } from 'react';

const TASKS = [
  { id: 1, title: 'Buy milk', done: true },
  { id: 2, title: 'Call Ana', done: false },
  { id: 3, title: 'Write the report', done: false },
];

function App() {
  // A hand-made URL in state; useSearchParams() would read the real one.
  const [url, setUrl] = useState('/tasks?done=false');
  const [pathname, query = ''] = url.split('?');
  const searchParams = new URLSearchParams(query);
  const done = searchParams.get('done');
  // → 'true', 'false' or null
  const q = searchParams.get('q') ?? '';

```

```
// what setSearchParams does
function update(key, value) {
  const next = new URLSearchParams(searchParams);
  if (value) next.set(key, value); else next.delete(key);
  const s = next.toString();
  setUrl(pathname + (s ? '?' + s : ''));
}

const shown = TASKS.filter((t) => (done === null || String(t.done) === done)
  && t.title.toLowerCase().includes(q.toLowerCase()));
return (
  <>
    <p>URL: <code>{url}</code></p>
    <label>Search <input value={q} onChange={(e) => update('q', e.target.value)} /></label>{' '}
    <select value={done ?? ''} onChange={(e) => update('done', e.target.value)} aria-label="Status">
      <option value="">All</option><option value="false">To do</option><option value="true">Done</option>
    </select>
    <ul>{shown.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
  </>
);
}
```

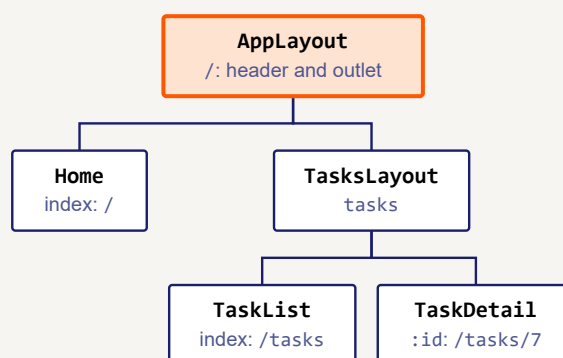
Example. In the Try it box, choose "All": the done key disappears from the URL and all three tasks show. Type "rep" in Search: the URL becomes `/tasks?q=rep` and the list follows. Every filter is in the URL, so a reload (or a shared link) would show exactly this list.

Common mistake. Writing the query string into the route path, `path="/tasks?done=:done"`. Route paths never include the query string (React Router matches only the path), so this route never matches. Keep `path="/tasks"` and read the filter with `useSearchParams`.

Nested routes and layouts

Nested routes and layouts: `<Outlet />`

Routes can contain routes: the parent route's element is a **layout** (header, navigation, sidebar) that renders `<Outlet />` where the matching child goes, and it stays mounted while the user moves between its children.



The route tree mirrors the screen: each layout draws the next level in its outlet.

- **Relative paths:** inside `<Route path="tasks">`, `path=":id"` means `/tasks/:id`.
- **Pathless layouts:** `<Route element={<AppLayout />}>` wraps its children in a layout (or a guard) without adding anything to their URLs.
- **The layout stays mounted:** its state (an open menu, a search box) survives navigation, and its effects do not run again.
- **Relative links:** in `/tasks/7`, `<Link to="edit">` goes to `/tasks/7/edit` and `to=".."` to the parent route.

```
// App.jsx: the route tree mirrors the screen tree
<Routes>
  <Route path="/" element={<AppLayout />}>
```

```

    <Route index element={<Home />} />
    <Route path="tasks" element={<TasksLayout />}>
      <Route index element={<TaskList />} />
      <Route path=":id" element={<TaskDetail />} />
    </Route>
  </Route>
</Routes>

// AppLayout.jsx: drawn on every page
import { Outlet, NavLink } from 'react-router';

export default function AppLayout() {
  return (
    <>
      <header>
        <nav>
          <NavLink to="/" end>Home</NavLink>
          <NavLink to="/tasks">Tasks</NavLink>
        </nav>
      </header>
      <main>
        <Outlet />          {/* the matching child route renders here */}
      </main>
    </>
  );
}

```

```

import { useState } from 'react';

// Hand-made nesting: the layout receives the child page as "outlet"
// (in React Router it would write <Outlet /> instead).
function AppLayout({ outlet, go }) {
  const [note, setNote] = useState('');
  return (
    <div style={{ border: '2px solid teal', padding: 8 }}>
      <header>
        <strong>Tasks app</strong>{' '}
        <button onClick={() => go('/')}>Home</button>
        <button onClick={() => go('/tasks')}>Tasks</button>
        <button onClick={() => go('/tasks/2')}>Task 2</button>
        <p><input placeholder="Type here, then navigate" value={note}
          onChange={(e) => setNote(e.target.value)} /></p>
      </header>
      <main style={{ border: '2px dashed orange', padding: 8 }}>{outlet}</main>
    </div>
  );
}

function App() {
  const [path, setPath] = useState('/tasks');
  let page = <p>Not found</p>;
  if (path === '/') page = <h2>Home</h2>;
  else if (path === '/tasks') page = <h2>All tasks</h2>;
  else if (path.startsWith('/tasks/')) page = <h2>Task {path.split('/')[2]}</h2>;
  return <AppLayout outlet={page} go={setPath} />;
}

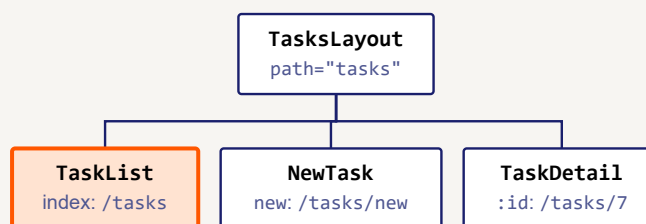
```

Example. In the Try it box, type something in the header's input, then click Home, Tasks and Task 2: only the dashed area (the outlet) changes and your text stays, because AppLayout is never unmounted. If every page rendered its own copy of the header, the input would be recreated, and emptied, on every click.

Common mistake. Nesting the routes but forgetting <Outlet /> in the parent's element. The URL changes, the parent renders, and the child silently does not appear: there is no error, the router simply has nowhere to put it. If a child route "does not work", check that every element on the way down renders an outlet.

Index routes: the default child

An **index route**, `<Route index element={<TaskList />} />`, renders in its parent's outlet when the URL is exactly the parent's path: it has no path of its own.



The index child fills the outlet at the parent's own URL.

- **A ranking bonus** of +2, so at `/tasks` "layout + index" wins over the layout alone.
- **A leaf**: no children and no path of its own; usually written first, for readability.
- **At the root**: `<Route index element={<Home />} />` inside the root layout gives the home page its header and navigation.
- **A redirect as the index**: `<Route index element={<Navigate to="tasks" replace />} />` makes `/tasks` the real start page. `<Navigate>` is a component that navigates as soon as it renders (see [Navigating from code](#), and for `replace`, [Push or replace](#)).

```
<Route path="tasks" element={<TasksLayout />}>
  <Route index element={<TaskList />} />    { /* /tasks */ }
  <Route path="new" element={<NewTask />} />  { /* /tasks/new */ }
  <Route path=":id" element={<TaskDetail />} /> { /* /tasks/7 */ }
</Route>
```

Example. With these routes, `/tasks` renders `TasksLayout` with `TaskList` in the outlet. Remove the index route and `/tasks` still matches (the parent has a path), but the outlet is empty: the layout's title and buttons with a blank area below.

Common mistake. Giving the index route the parent's path again, `<Route path="tasks" element={<TaskList />} />` inside `<Route path="tasks">`. Child paths are relative, so this child lives at `/tasks/tasks`, and `/tasks` still has an empty outlet. Use `index`.

The catch-all route: a 404 page

A route with `path="*" matches any URL but has the lowest score, so it only renders when no other route matches: use it for a "Page not found" screen instead of a blank page.`

- **Position does not matter**: `*` has the lowest score. Put it **inside** the layout route, so the header and navigation stay on the not-found page.
- **Under a path**: `path="docs/*"` matches everything below `/docs`; the rest of the URL is in `useParams()['*']`.
- **Missing data is another 404**: `/tasks/999` matches `/tasks/:id`, so the catch-all cannot help; the page must handle the API's 404 (see [Loading data for a route](#)). The server makes the same distinction (see [The 404 catch-all](#)).

```
<Routes>
  <Route path="/" element={<AppLayout />}>
    <Route index element={<Home />} />
    <Route path="tasks" element={<TaskList />} />
    <Route path="tasks/:id" element={<TaskDetail />} />
    <Route path="*" element={<NotFound />} />    { /* last resort, inside the layout */ }
  </Route>
</Routes>

function NotFound() {
  const location = useLocation();
  return (
    <section>
```

```

    <h1>Page not found</h1>
    <p>Nothing lives at <code>{location.pathname}</code>.</p>
    <Link to="/tasks">Back to your tasks</Link>
  </section>
);
}

```

Example. With these routes, `/tsks` and `/tasks/7/oops` both render `AppLayout` with `NotFound` in its outlet: neither matches a real route (`/tasks/:id` takes exactly one segment after `tasks`), so only `*` is left.

Common mistake. Redirecting unknown URLs to the home page with `<Route path="*" element={<Navigate to="/" />} />`. The user who mistyped never learns that the page does not exist, and a broken link in your own app becomes impossible to notice. Show a real not-found page.

Navigating from code

Navigating from code: `useNavigate`

`const navigate = useNavigate()` gives a function that changes the URL from code (`navigate('/tasks/12')` after a save, `navigate('/login')` after logging out, `navigate(-1)` for Back): use it when the navigation is the result of an action, not a place the user clicks.

`navigate('/tasks')`, `navigate('edit')`,
`navigate('..')`

The same targets as a `<Link>`: absolute, or relative to the current route.

`navigate(-1)`

A number moves through the history: `-1` is Back.

`{ replace: true }`

Second argument: overwrite the current history entry instead of adding one.

`{ state: { ... } }`

Second argument: data attached to the new entry, read on the next page with `useLocation().state`.

```

import { useState } from 'react';

// Hand-made: the history is an array of URLs; navigate() adds one (push)
// or overwrites the last one (replace). This toy Back forgets the forward page.
function App() {
  const [stack, setStack] = useState(['/', '/tasks/new']);
  const [tasks, setTasks] = useState([{ id: 1, title: 'Buy milk' }]);
  const [title, setTitle] = useState('');
  const path = stack[stack.length - 1];
  const navigate = (to, { replace = false } = {}) =>
    setStack((s) => (replace ? [...s.slice(0, -1), to] : [...s, to]));

  function handleSubmit(e) {
    e.preventDefault();
    const task = { id: tasks.length + 1, title };
    setTasks([...tasks, task]);
    setTitle('');
    // try { replace: true }
    navigate('/tasks/' + task.id);
  }

  let page = <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>;
  if (path === '/tasks/new') page = (
    <form onSubmit={handleSubmit}>
      <label>Title <input value={title} onChange={(e) => setTitle(e.target.value)} required /></label>
      <button>Save</button>
    </form>
  );
  else if (path !== '/tasks') page = <h2>{tasks.find((t) => '/tasks/' + t.id === path)?.title}</h2>;
  return (
    <>
      <p><button onClick={() => setStack((s) => s.slice(0, -1))} disabled={stack.length < 2}>Back</button>{' '}
        <code>{path}</code></p>
    </>
  );
}

```

```

    {page}
    <p>History: {stack.join(' → ')}</p>
  </>
);
}

```

Example. In the Try it box, save a task called "Call Ana": the app shows it at `/tasks/2` and the history is `/tasks → /tasks/new → /tasks/2`. Press Back: you land on the empty form again, which is rarely what users want. Change the call to `navigate('/tasks/' + task.id, { replace: true })` and repeat: the form's entry is overwritten, and Back goes straight to the list.

Common mistake. Navigating before the work is done: `api.createTask(values); navigate('/tasks');` without `await`. The list page mounts and fetches while the POST is still on its way, so the new task is missing, and if the POST fails the user never sees the error. `await` the request, check it succeeded, then navigate.

Push or replace: what the Back button will do

Every navigation either **pushes** a new history entry (Back returns to the current page) or **replaces** the current one (Back skips it); use `replace` for pages the user should not come back to: redirects, the log-in page after logging in, a form after it was saved.

History after logging in: the user started on `/tasks` and clicked Settings

Guard	After log-in	History	Back from /settings goes to
push	push	<code>/tasks → /settings → /login → /settings</code>	<code>/login</code> (the form, already logged in)
replace	push	<code>/tasks → /login → /settings</code>	<code>/login</code>
push	replace	<code>/tasks → /settings → /settings</code>	<code>/settings</code> (nothing seems to happen)
replace	replace	<code>/tasks → /settings</code>	<code>/tasks</code>

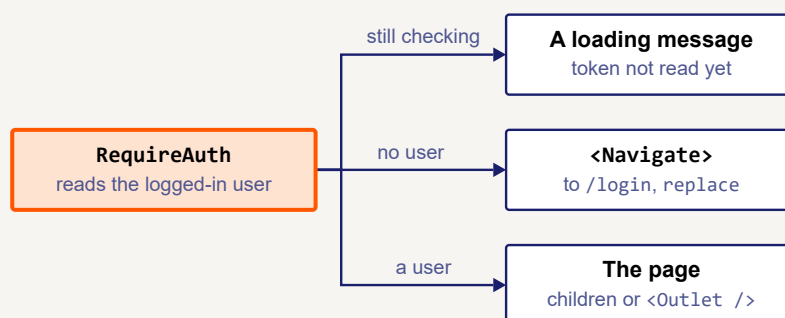
Example. In the router simulator, open challenge 8: both switches start off. Click Settings while logged out, log in, then press Back and watch the history stack: you land on `/login`. Switch on the guard's `replace` and the log-in `replace`, run the story again, and Back returns to `/tasks`.

Common mistake. Using `replace` everywhere "to keep the history clean". Then Back skips pages the user did want: from a task's detail, Back leaves the app instead of returning to the list. Push is the default for a reason; replace only the entries that should not be revisited.

Protected routes and data

Protected routes: a guard component

A **protected route** renders its page only for a logged-in user: a small wrapper component, often called `RequireAuth`, renders the page (or `<Outlet />`), or `<Navigate to="/login" replace />` when nobody is logged in.



The guard decides what renders: a wait, a redirect or the page.

- **Around one page:** `element={<RequireAuth><Settings /></RequireAuth>}`.
- **Around a group:** a pathless layout route, `<Route element={<RequireAuth />}>...</Route>`, whose guard renders `<Outlet />`.

- **Never guard** /login: a logged-out user would be sent from /login to /login forever.
- **User experience, not security:** anyone can bypass front-end code or call the API directly. The server protects the data by checking the token on every request (see [Authentication and security](#)).

```
// RequireAuth.jsx
import { Navigate, Outlet, useLocation } from 'react-router';
import { useAuth } from './AuthContext.jsx';

export default function RequireAuth({ children }) {
  const { user, checking } = useAuth();
  const location = useLocation();
  // still reading the saved token
  if (checking) return <p>Loading...</p>;
  if (!user) {
    return <Navigate to="/login" replace state={{ from: location }} />;
  }
  // wraps one page, or a group of child routes
  return children ?? <Outlet />;
}

// App.jsx
<Route path="/" element={<AppLayout />}>
  <Route index element={<Home />} />
  <Route path="login" element={<Login />} />      {/* public, never guarded */}
  <Route element={<RequireAuth />}                {/* every route inside is private */}
    <Route path="tasks/new" element={<NewTask />} />
    <Route path="tasks/:id/edit" element={<EditTask />} />
    <Route path="settings" element={<Settings />} />
  </Route>
</Route>
```

```
import { useEffect, useState } from 'react';

// Hand-made pieces that imitate React Router: the location in state,
// a <Navigate> that redirects once it renders, and a guard.
function Navigate({ to, state, navigate }) {
  useEffect(() => { navigate(to, { state }); }, [to]);
  return null;
}

function RequireAuth({ user, location, navigate, children }) {
  if (!user) return <Navigate to="/login" state={{ from: location.path }} navigate={navigate} />;
  return children;
}

function App() {
  const [location, setLocation] = useState({ path: '/', state: null });
  const [user, setUser] = useState(null);
  const navigate = (path, opts = {}) => setLocation({ path, state: opts.state ?? null });

  let page = <h2>Home (public)</h2>;
  if (location.path === '/settings') page = (
    <RequireAuth user={user} location={location} navigate={navigate}>
      <h2>Settings of {user?.name}</h2>
    </RequireAuth>
  );
  if (location.path === '/login') page = (
    <button onClick={() => { setUser({ name: 'Ana' }); navigate(location.state?.from ?? '/'); }}>
      Log in as Ana
    </button>
  );
  return (
    <>
      <nav>
        <button onClick={() => navigate('/')}>Home</button>{' '}
        <button onClick={() => navigate('/settings')}>Settings</button>{' '}
        {user && <button onClick={() => setUser(null)}>Log out</button>}
      </nav>
    </>
  );
}
```

```

    </nav>
    <p>URL <code>{location.path}</code> · state <code>{JSON.stringify(location.state)}</code> · user: {user ?
user.name : 'nobody'}</p>
    {page}
  </>
);
}

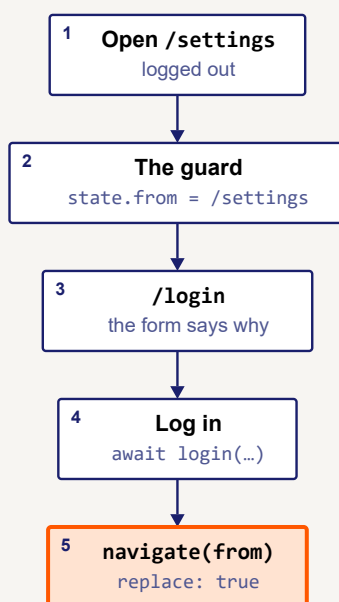
```

Example. In the Try it box, click Settings while logged out: the guard renders the redirect, the URL becomes `/login` and the state remembers `{"from": "/settings"}`. Log in: you land on Settings. Click Log out while on Settings: the guard runs again and sends you back to `/login`.

Common mistake. Starting the auth state as "logged out" and restoring the saved token later, in an effect: on a refresh of `/settings` the guard redirects before the token is read, so logged-in users are thrown out on every reload. Read the token in the initial state, or keep a checking flag as the code does (see [Staying logged in after a reload](#)).

Back to where you were after log-in

When the guard redirects, it stores the page the user wanted in the navigation **state** (`<Navigate to="/login" replace state={{ from: location }} />`); after a successful log-in, the log-in page reads `useLocation().state?.from` and navigates there, with `replace`.



The guard writes down the destination; the log-in page reads it back.

- **History state** is data attached to a history entry, invisible in the URL: it survives a refresh of that entry, but not a typed URL or a shared link. Always keep a fallback: `location.state?.from?.pathname ?? '/'`.
- **Keep the query string:** `from.pathname + from.search`, so `/tasks?page=3` does not come back as `/tasks`.
- **Navigate after success:** `await login(email, password)` (see [Logging in from the front end](#)), then `navigate(from, { replace: true })`; on failure, stay and show the error.
- **Where the pieces come from:** the form is read with `new FormData(e.currentTarget)` (see [Uncontrolled inputs](#)); login comes from wherever the app keeps the logged-in user (see [An authentication context](#)).

```

// Login.jsx
import { useState } from 'react';
import { useLocation, useNavigate } from 'react-router';
import { useAuth } from './AuthContext.jsx';

export default function Login() {

```

```

const { login } = useAuth();
const navigate = useNavigate();
const location = useLocation();
const from = location.state?.from;
const target = from ? from.pathname + from.search : '/tasks';
const [error, setError] = useState(null);

async function handleSubmit(e) {
  e.preventDefault();
  const form = new FormData(e.currentTarget);
  try {
    await login(form.get('email'), form.get('password'));
    // back where they wanted to go
    navigate(target, { replace: true });
  } catch (err) {
    // stay here and explain
    setError(err.message);
  }
}

return (
  <form onSubmit={handleSubmit}>
    {from && <p>Log in to see {from.pathname}</p>}
    {/* email and password fields, the error, a submit button */}
  </form>
);
}

```

Example. A colleague sends a link to `/tasks/12/edit`. Logged out, the user opens it: the guard redirects to `/login` with `state.from = { pathname: '/tasks/12/edit', search: '' }`, and the form says "Log in to see `/tasks/12/edit`". After logging in, `navigate('/tasks/12/edit', { replace: true })` opens the edit page, and Back leaves for the page they came from, not the log-in form.

Common mistake. Putting the destination in the query string, `/login?next=/tasks/12`, and navigating to it without checking. Anyone can then craft a log-in link that sends users to `?next=https://evil.example` after logging in (an "open redirect"). Prefer history state, or accept only paths that start with a single `/`.

Loading data for a route

A page for one resource reads the param (`useParams()`) and fetches that resource in an effect with the param in its dependencies, `useEffect(..., [id])`: moving from `/tasks/1` to `/tasks/2` keeps the same component mounted, so only the dependency makes it fetch again.

- **Each page loads its own data:** the detail page must also work when opened from a bookmark, so the list does not hand it the task.
- **Handle the answers that belong to a URL:** a 404 deserves "This task does not exist" and a link to the list; a 400 usually means the param is not a valid id.
- **The rest of the request:** the loading, error, empty and success states in [Loading](#), [error](#), [empty](#), [success](#); a late answer for the old id in [Race conditions](#).
- **Data mode:** a route loader (`loader: ({ params }) => ...`, read with `useLoaderData()`) starts the fetch before the page renders; the rules about ids and errors are the same.

```

import { useEffect, useState } from 'react';

// in React Router: const { id } = useParams();
function TaskDetail({ id }) {
  const [task, setTask] = useState(null);
  const [error, setError] = useState(null);
  useEffect(() => {
    let ignore = false;
    setTask(null);
    setError(null);
    fetch('/api/tasks/' + id)
      .then(async (res) => {
        const body = await res.json();
        if (!res.ok) throw new Error(res.status === 404 ? 'This task does not exist.' : body.error);
      })
      .catch(() => {
        // ...
      })
  }, [id]);
}

```

```

        return body;
    })
    .then((t) => { if (!ignore) setTask(t); })
    .catch((e) => { if (!ignore) setError(e.message); });
    // a newer id arrived: ignore this answer
    return () => { ignore = true; };
    // a new id in the URL → fetch again
  }, [id]);
  if (error) return <p role="alert">Task {id}: {error}</p>;
  if (!task) return <p>Loading task {id}...</p>;
  return <h2>#{task.id} {task.title} {task.done ? '(done)' : ''}</h2>;
}

function App() {
  const [path, setPath] = useState('/tasks/1');
  const id = path.split('/')[2];
  return (
    <>
      <nav style={{ display: 'flex', flexWrap: 'wrap', gap: 6 }}>
        {[ '1', '2', '3', '999', 'abc' ].map((n) => (
          <button key={n} onClick={() => setPath('/tasks/' + n)}>/tasks/{n}</button>
        ))}
      </nav>
      <p>URL: <code>{path}</code></p>
      <TaskDetail id={id} />
    </>
  );
}

```

Example. In the Try it box, click through /tasks/1, /tasks/2 and /tasks/3: each click shows "Loading..." then the task, because id changed and the effect ran again (the console lists one request per click). /tasks/999 shows "This task does not exist." (the API answered 404) and /tasks/abc the API's validation error (400). Change [id] to [] and click around: the URL changes, the task does not.

Common mistake. Fetching all tasks on the detail page and picking one, tasks.find((t) => t.id === id), or relying on the list page to pass the task in navigation state. The first downloads everything to show one item (and compares a number with the string param); the second breaks as soon as someone opens /tasks/7 from a bookmark. Fetch the one resource the URL names.